Step 1 :- Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_csv(r"D:\INTERNSHIP PROJECT\dataset\train.csv")

# ----------------------------
# Basic Dataset Information
# ----------------------------
print("\n========== FIRST 5 ROWS ==========\n")
print(df.head())

print("\n========== DATASET SHAPE ==========\n")
print(df.shape)

print("\n========== COLUMN NAMES ==========\n")
print(df.columns)

print("\n========== DATASET INFORMATION ==========\n")
df.info()

print("\n========== STATISTICAL SUMMARY ==========\n")
print(df.describe())

print("\n========== MISSING VALUES ==========\n")
print(df.isnull().sum())

print("\n========== DUPLICATE RECORDS ==========\n")
print(df.duplicated().sum())


# ============================================
# EDA (Exploratory Data Analysis)
# ============================================

# 1 Placement Status Distribution
plt.figure(figsize=(6,5))
sns.countplot(x='Placement_Status', data=df)
plt.title("Placement Status Distribution")
plt.xlabel("Placement Status")
```

```
plt.ylabel("Number of Students")
plt.show()

# 2 Gender Distribution
plt.figure(figsize=(6,5))
sns.countplot(x='Gender', data=df)
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()

# 3 Degree Distribution
plt.figure(figsize=(8,5))
sns.countplot(x='Degree', data=df)
plt.title("Degree Distribution")
plt.xlabel("Degree")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()

# 4 Branch Distribution
plt.figure(figsize=(12,5))
sns.countplot(x='Branch', data=df)
plt.title("Branch Distribution")
plt.xlabel("Branch")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()

# 5 CGPA Distribution
plt.figure(figsize=(8,5))
sns.histplot(df['CGPA'], bins=20, kde=True)
plt.title("CGPA Distribution")
plt.xlabel("CGPA")
plt.ylabel("Frequency")
plt.show()

# 6 Age Distribution
plt.figure(figsize=(8,5))
sns.histplot(df['Age'], bins=10, kde=True)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
```

```
plt.show()
```

Output ➖

```
========== FIRST 5 ROWS ==========

   Student_ID  Age  Gender  Degree  ... Soft_Skills_Rating  Certifications  Backlogs
Placement_Status
0        1048   22  Female  B.Tech  ...                 5               1         3       Not Placed
1       37820   20  Female     BCA  ...                 8               2         1       Not Placed
2       49668   22    Male     MCA  ...                 6               2         2       Not Placed
3       19467   22    Male     MCA  ...                 4               2         0           Placed
4       23094   20  Female  B.Tech  ...                 6               2         0           Placed

[5 rows x 15 columns]

========== DATASET SHAPE ==========

(45000, 15)

========== COLUMN NAMES ==========

Index(['Student_ID', 'Age', 'Gender', 'Degree', 'Branch', 'CGPA',
       'Internships', 'Projects', 'Coding_Skills', 'Communication_Skills',
       'Aptitude_Test_Score', 'Soft_Skills_Rating', 'Certifications',
       'Backlogs', 'Placement_Status'],
      dtype='str')

========== DATASET INFORMATION ==========

<class 'pandas.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 15 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Student_ID           45000 non-null  int64
 1   Age                  45000 non-null  int64
 2   Gender               45000 non-null  str
 3   Degree               45000 non-null  str
 4   Branch               45000 non-null  str
 5   CGPA                 45000 non-null  float64
 6   Internships          45000 non-null  int64
 7   Projects             45000 non-null  int64
 8   Coding_Skills        45000 non-null  int64
 9   Communication_Skills  45000 non-null  int64
 10  Aptitude_Test_Score   45000 non-null  int64
 11  Soft_Skills_Rating    45000 non-null  int64
 12  Certifications        45000 non-null  int64
```

```
 13  Backlogs             45000 non-null  int64
 14  Placement_Status     45000 non-null  str
dtypes: float64(1), int64(10), str(4)
memory usage: 5.1 MB

========== STATISTICAL SUMMARY ==========

        Student_ID           Age          CGPA  ...  Soft_Skills_Rating  Certifications      Backlogs
count  45000.000000  45000.000000  45000.000000  ...       45000.000000    45000.000000  45000.000000
mean   24977.962600     20.999333      7.002290  ...           5.501644        1.800956      0.888133
std    14425.605704      1.995071      0.993855  ...           1.238722        0.650104      0.970954
min        1.000000     18.000000      4.500000  ...           1.000000        0.000000      0.000000
25%    12509.750000     19.000000      6.320000  ...           5.000000        1.000000      0.000000
50%    24957.500000     21.000000      7.000000  ...           5.000000        2.000000      1.000000
75%    37475.250000     23.000000      7.670000  ...           6.000000        2.000000      2.000000
max    50000.000000     24.000000      9.800000  ...          10.000000        3.000000      3.000000

[8 rows x 11 columns]

========== MISSING VALUES ==========

Student_ID             0
Age                    0
Gender                 0
Degree                 0
Branch                 0
CGPA                   0
Internships            0
Projects               0
Coding_Skills          0
Communication_Skills   0
Aptitude_Test_Score    0
Soft_Skills_Rating     0
Certifications         0
Backlogs               0
Placement_Status       0
dtype: int64
```

Screenshots


Figure 1
Placement Status Distribution
30000
25000
20000
15000
10000
5000
0
Number of Students
Not Placed
Placed
Placement Status

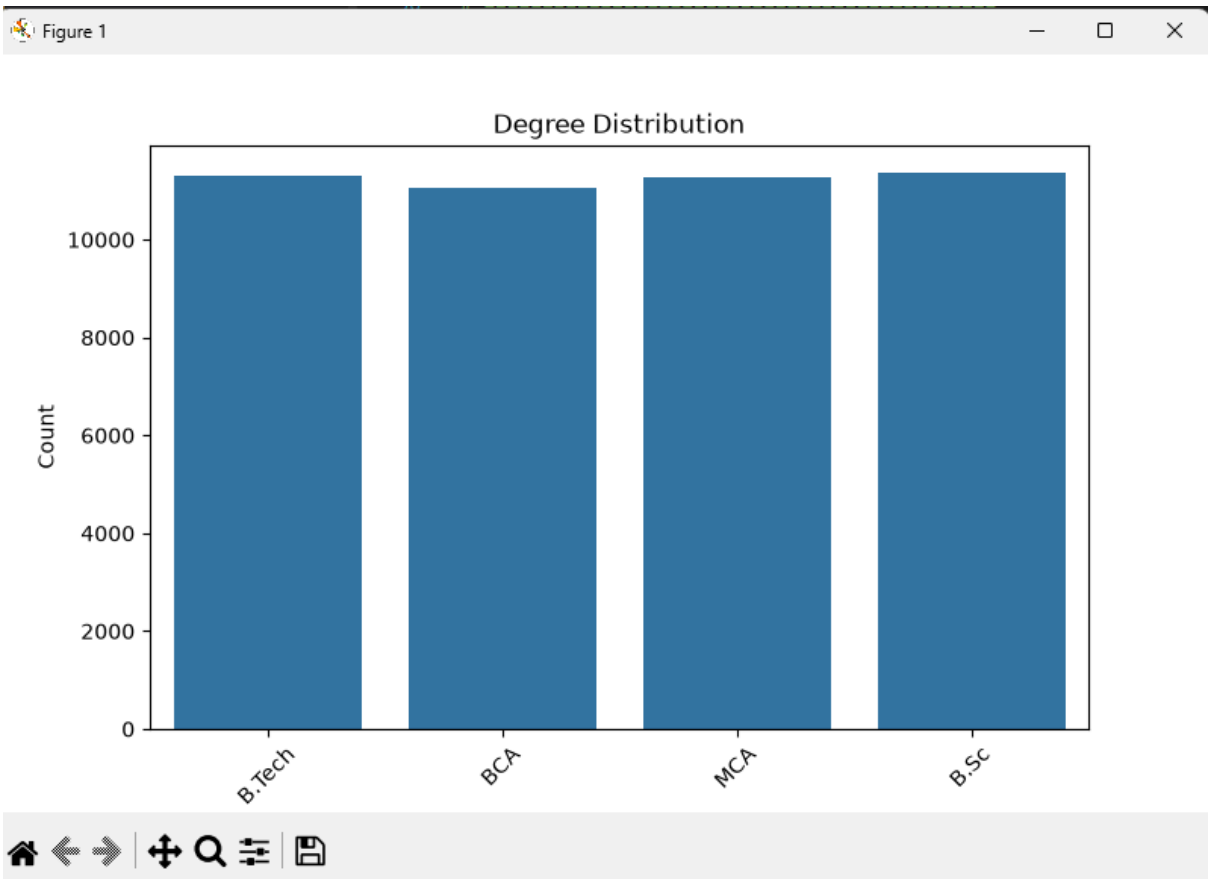


Figure 1
Degree Distribution
Count
10000
8000
6000
4000
2000
0
B.Tech
BCA
MCA
B.Sc

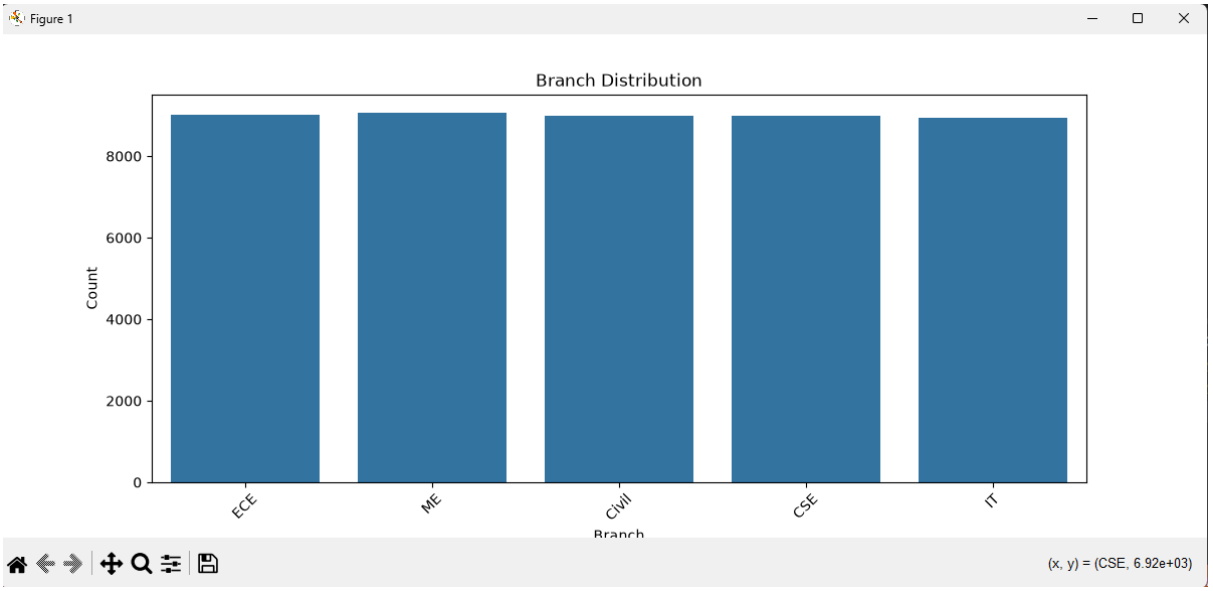
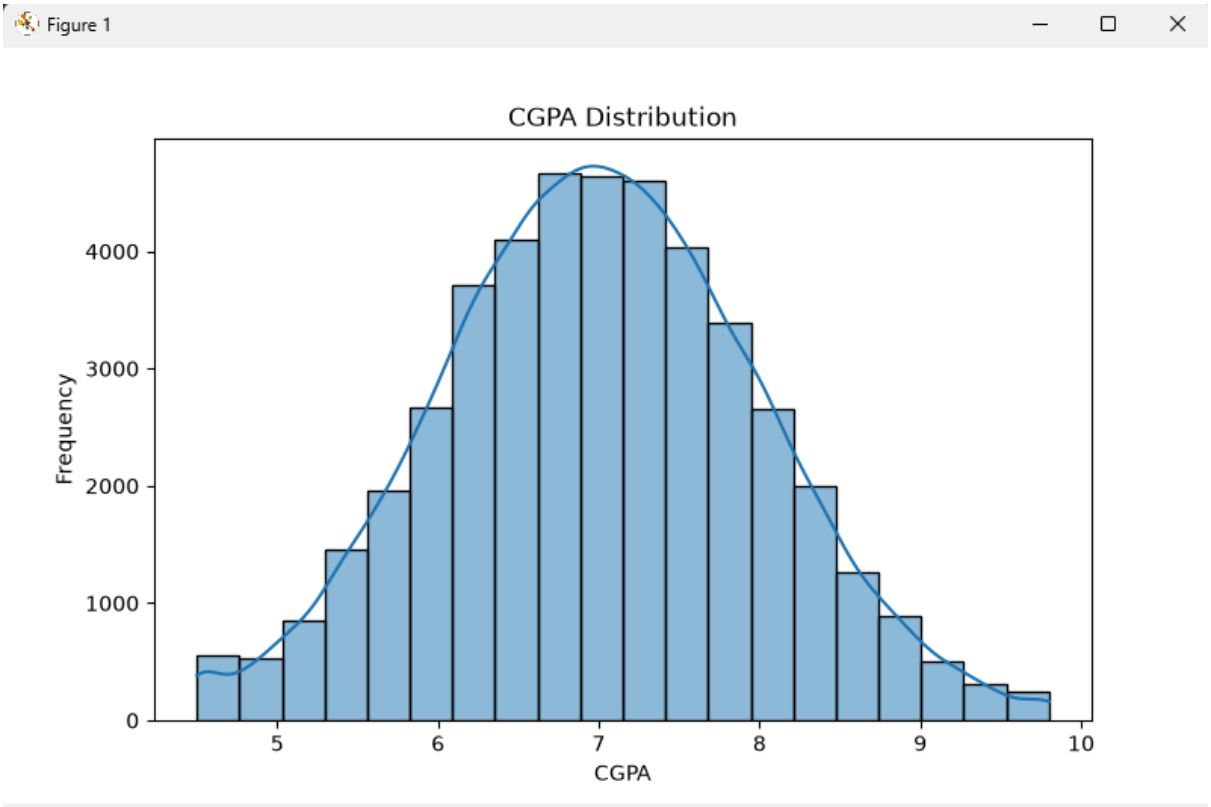
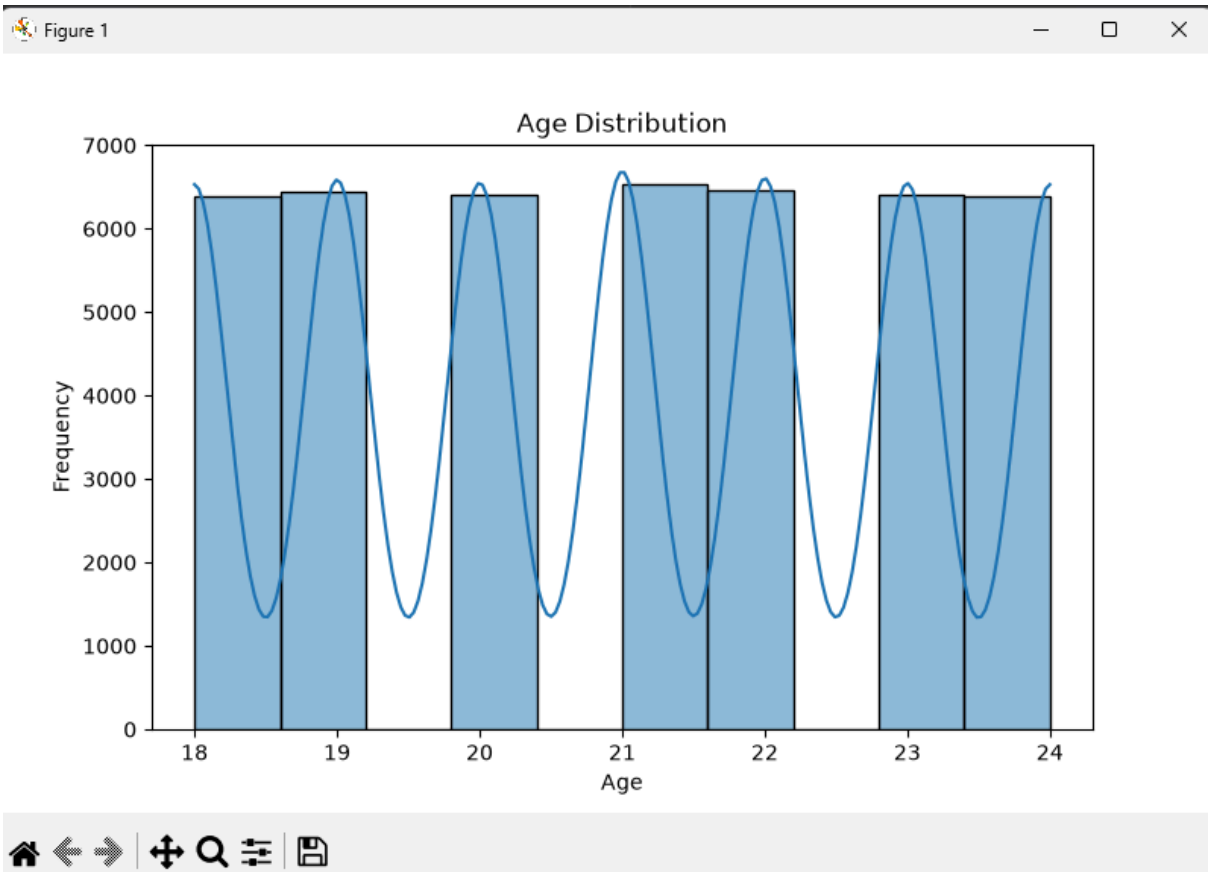


Figure 1
Branch Distribution
Count
8000
6000
4000
2000
0
ECE
ME
Civil
CSE
IT
Branch
(x, y) = (CSE, 6.92e+03)





Step 2 :- # ==========

```
# STEP 2 : EXPLORATORY DATA ANALYSIS (EDA)
```

```python
# Project : Smart Campus Placement Prediction System
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_csv(r"D:\INTERNSHIP PROJECT\dataset\train.csv")

sns.set_style("whitegrid")

# ============================================================
# 1. Internships Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.countplot(x='Internships', data=df)
plt.title("Internships Distribution")
plt.xlabel("Number of Internships")
plt.ylabel("Number of Students")
plt.show()

# ============================================================
# 2. Projects Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.countplot(x='Projects', data=df)
plt.title("Projects Distribution")
plt.xlabel("Projects")
plt.ylabel("Number of Students")
plt.show()

# ============================================================
# 3. Coding Skills Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.histplot(df['Coding_Skills'], bins=10, kde=True)
plt.title("Coding Skills Distribution")
plt.xlabel("Coding Skills")
plt.ylabel("Frequency")
plt.show()

# ============================================================
```

```
# 4. Communication Skills Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.histplot(df['Communication_Skills'], bins=10, kde=True)
plt.title("Communication Skills Distribution")
plt.xlabel("Communication Skills")
plt.ylabel("Frequency")
plt.show()

# ============================================================
# 5. Aptitude Test Score Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.histplot(df['Aptitude_Test_Score'], bins=10, kde=True)
plt.title("Aptitude Test Score Distribution")
plt.xlabel("Aptitude Test Score")
plt.ylabel("Frequency")
plt.show()

# ============================================================
# 6. Soft Skills Rating Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.histplot(df['Soft_Skills_Rating'], bins=10, kde=True)
plt.title("Soft Skills Rating Distribution")
plt.xlabel("Soft Skills Rating")
plt.ylabel("Frequency")
plt.show()

# ============================================================
# 7. Certifications Distribution
# ============================================================
plt.figure(figsize=(7,5))
sns.countplot(x='Certifications', data=df)
plt.title("Certifications Distribution")
plt.xlabel("Certifications")
plt.ylabel("Number of Students")
plt.show()

# ============================================================
# 8. Backlogs Distribution
# ============================================================
plt.figure(figsize=(7,5))
```

```
sns.countplot(x='Backlogs', data=df)
plt.title("Backlogs Distribution")
plt.xlabel("Backlogs")
plt.ylabel("Number of Students")
plt.show()

# ============================================================
# 9. Placement Status vs Gender
# ============================================================
plt.figure(figsize=(7,5))
sns.countplot(x='Gender', hue='Placement_Status', data=df)
plt.title("Placement Status vs Gender")
plt.show()

# ============================================================
# 10. Placement Status vs Degree
# ============================================================
plt.figure(figsize=(8,5))
sns.countplot(x='Degree', hue='Placement_Status', data=df)
plt.xticks(rotation=45)
plt.title("Placement Status vs Degree")
plt.show()

# ============================================================
# 11. Placement Status vs Branch
# ============================================================
plt.figure(figsize=(12,5))
sns.countplot(x='Branch', hue='Placement_Status', data=df)
plt.xticks(rotation=45)
plt.title("Placement Status vs Branch")
plt.show()

# ============================================================
# 12. CGPA vs Placement (Boxplot)
# ============================================================
plt.figure(figsize=(7,5))
sns.boxplot(x='Placement_Status', y='CGPA', data=df)
plt.title("CGPA vs Placement Status")
plt.show()

# ============================================================
# 13. Correlation Heatmap
# ============================================================
```

```
numeric_df = df.select_dtypes(include=['number'])

plt.figure(figsize=(10,8))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

Output :-

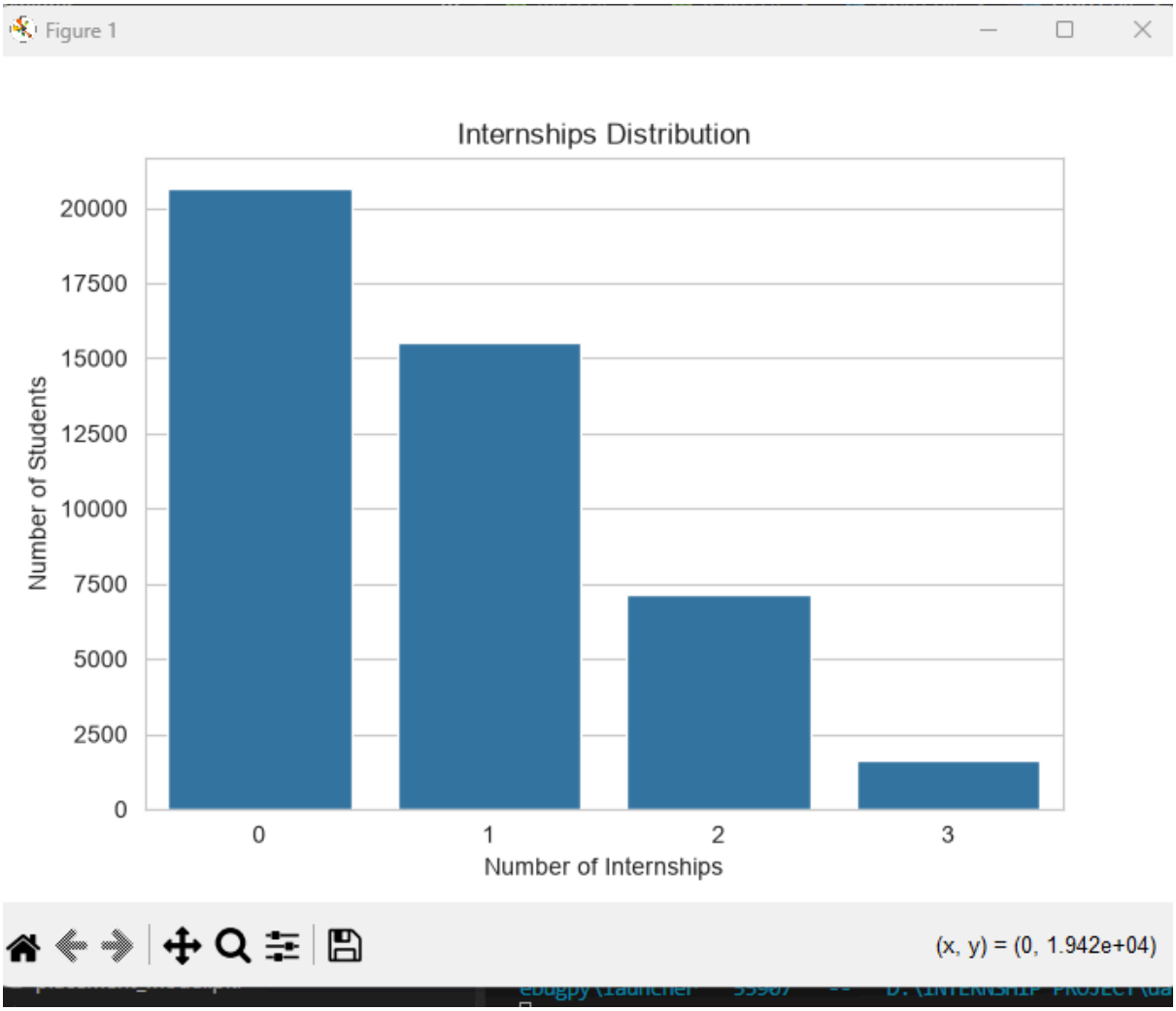
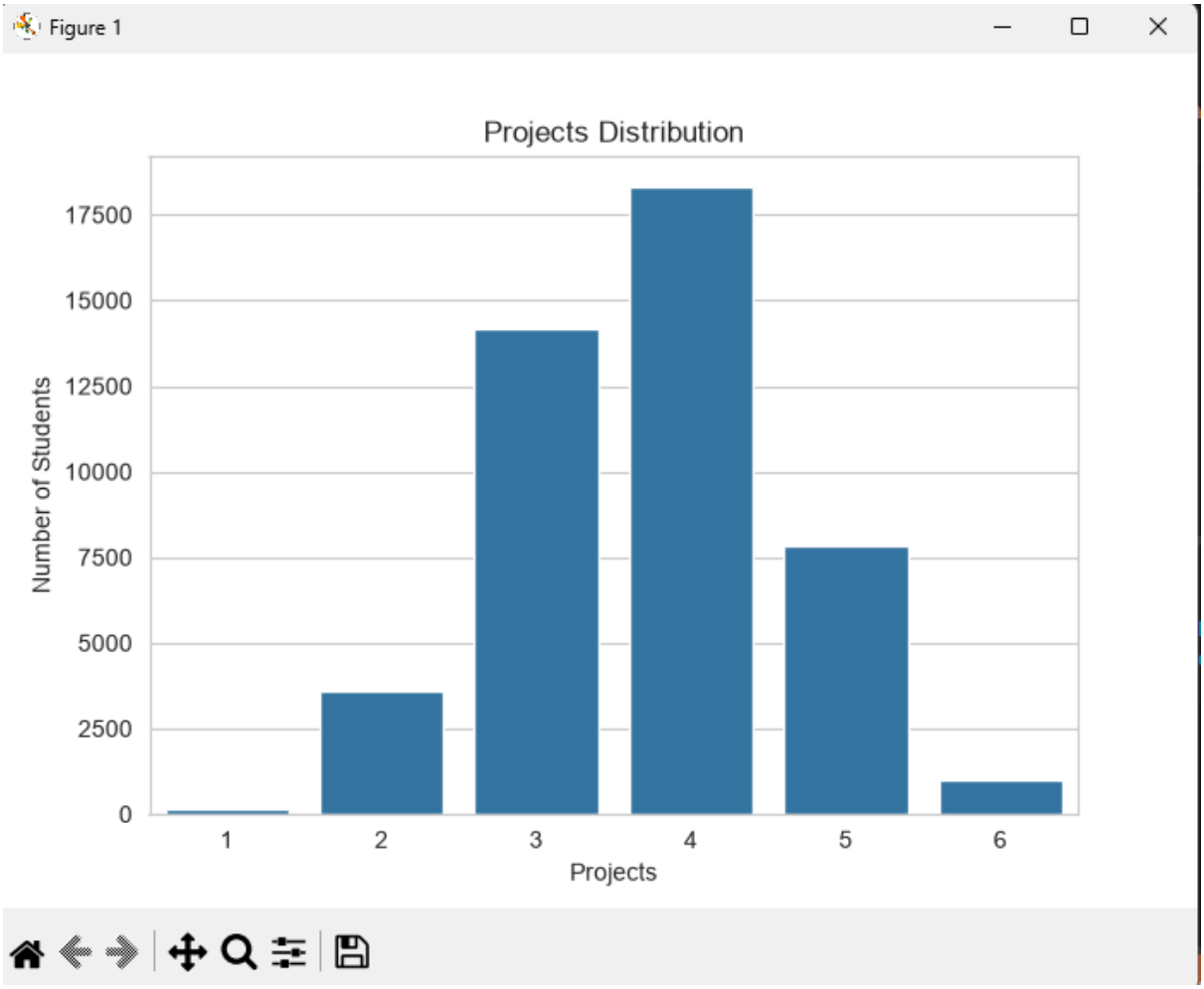


Figure 1
Internships Distribution
Number of Students
20000
17500
15000
12500
10000
7500
5000
2500
0
0
1
2
3
Number of Internships
(x, y) = (0, 1.942e+04)



Step 3

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
import pickle

# Load dataset
df = pd.read_csv(r"D:\INTERNSHIP PROJECT\dataset\train.csv")

print(df.head())
print(df.isnull().sum())

# ----------------------------
# Handle Missing Values
# ----------------------------
numeric_columns = df.select_dtypes(include=['int64',
'float64']).columns

for col in numeric_columns:
```

```
    df[col] = df[col].fillna(df[col].mean())

categorical_columns = df.select_dtypes(include=['object']).columns

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# ---------------------------
# Remove Duplicates
# ---------------------------
print("Duplicates before:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Duplicates after:", df.duplicated().sum())

# ---------------------------
# Label Encoding (FIXED)
# ---------------------------
encoders = {}

for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le   # save encoder for each column

# Save encoders
pickle.dump(encoders, open("encoders.pkl", "wb"))

print(df.head())

# ---------------------------
# Split Features & Target
# ---------------------------
X = df.drop("Placement_Status", axis=1)
y = df["Placement_Status"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(X_train.shape)
print(X_test.shape)

# ---------------------------
```

```
# Feature Scaling
# ---------------------------
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Save scaler
pickle.dump(scaler, open("scaler.pkl", "wb"))

print("✅ Step 3 Completed Successfully")
df = pd.read_csv(r"D:\INTERNSHIP PROJECT\dataset\train.csv")

print(df.columns.tolist())
```

Output ➖

	Student_ID	Age	Gender	Degree	...	Soft_Skills_Rating	Certifications	Backlogs	Placement_Status
0	1048	22	Female	B.Tech	...	5	1	3	Not Placed
1	37820	20	Female	BCA	...	8	2	1	Not Placed
2	49668	22	Male	MCA	...	6	2	2	Not Placed
3	19467	22	Male	MCA	...	4	2	0	Placed
4	23094	20	Female	B.Tech	...	6	2	0	Placed

[5 rows x 15 columns]

```
Student_ID              0
Age                     0
Gender                  0
Degree                  0
Branch                  0
CGPA                    0
Internships             0
Projects                0
Coding_Skills           0
Communication_Skills    0
Aptitude_Test_Score     0
Soft_Skills_Rating      0
Certifications          0
Backlogs                0
Placement_Status        0
dtype: int64
```

D:\INTERNSHIP PROJECT\dataset\step3.py:20: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical_columns = df.select_dtypes(include=['object']).columns
Duplicates before: 0
Duplicates after: 0
   Student_ID Age Gender Degree ... Soft_Skills_Rating Certifications Backlogs Placement_Status
0   1048   22   0   1 ...   5   1   3   0
1   37820   20   0   2 ...   8   2   1   0
2   49668   22   1   3 ...   6   2   2   0
3   19467   22   1   3 ...   4   2   0   1
4   23094   20   0   1 ...   6   2   0   1

[5 rows x 15 columns]
(36000, 14)
(9000, 14)
✅ Step 3 Completed Successfully
['Student_ID', 'Age', 'Gender', 'Degree', 'Branch', 'CGPA', 'Internships', 'Projects', 'Coding_Skills', 'Communication_Skills', 'Aptitude_Test_Score', 'Soft_Skills_Rating', 'Certifications', 'Backlogs', 'Placement_Status']

Step 4 ➖

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import pickle

# Load data
df = pd.read_csv(r"D:\INTERNSHIP PROJECT\dataset\train.csv")

# Same preprocessing
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Fill missing values
numeric_columns = df.select_dtypes(include=['int64',
'float64']).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].mean())

categorical_columns = df.select_dtypes(include=['object']).columns
```

```
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# Remove duplicates
df.drop_duplicates(inplace=True)

# Encoding
encoders = {}

for col in categorical_columns:
    encoder = LabelEncoder()
    df[col] = encoder.fit_transform(df[col])
    encoders[col] = encoder

# Input and output
X = df.drop("Placement_Status", axis=1)
y = df["Placement_Status"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

# Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = LogisticRegression()

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", accuracy)
```

```
print("Accuracy Percentage =", accuracy * 100, "%")

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Save files
pickle.dump(model, open("placement_model.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
pickle.dump(encoders, open("encoders.pkl", "wb"))

print("\nModel Saved Successfully")
```

Output ➖
Accuracy = 0.8633333333333333
Accuracy Percentage = 86.33333333333333 %

Index.html ➖

```
<!DOCTYPE html>
<html>
<head>
    <title>Smart Campus Placement Predictor</title>

    <style>
        body {
            font-family: Arial;
            background: linear-gradient(135deg, #36b9f0, #14d9d9);
            margin: 0;
            padding: 40px;
        }

        .container {
            width: 600px;
            margin: auto;
            background: white;
            padding: 30px;
            border-radius: 15px;
        }
```

```
        h1 {
            text-align: center;
        }

        label {
            display: block;
            margin-top: 12px;
            font-weight: bold;
        }

        input, select {
            width: 100%;
            padding: 10px;
            margin-top: 5px;
            box-sizing: border-box;
        }

        button {
            width: 100%;
            padding: 12px;
            margin-top: 20px;
            background: #2196f3;
            color: white;
            border: none;
            border-radius: 5px;
            font-size: 16px;
        }

        .result {
            text-align: center;
            margin-top: 20px;
            font-size: 24px;
            font-weight: bold;
        }
    </style>
</head>

<body>

<div class="container">

    <h1>Smart Campus Placement Predictor</h1>
```

```
    <form action="/predict" method="POST">

        <label>Student ID</label>
        <input type="number" name="Student_ID" required>

        <label>Age</label>
        <input type="number" name="Age" required>

        <label>Gender</label>
        <select name="Gender" required>
            <option value="">Select Gender</option>
            <option value="Male">Male</option>
            <option value="Female">Female</option>
        </select>

        <label>Degree</label>
        <input type="text" name="Degree" placeholder="B.Tech" required>

        <label>Branch</label>
        <input type="text" name="Branch" placeholder="CSE" required>

        <label>CGPA</label>
        <input type="number" step="0.01" name="CGPA" required>

        <label>Internships</label>
        <input type="number" name="Internships" required>

        <label>Projects</label>
        <input type="number" name="Projects" required>

        <label>Coding Skills</label>
        <input type="number" name="Coding_Skills" required>

        <label>Communication Skills</label>
        <input type="number" name="Communication_Skills" required>

        <label>Aptitude Test Score</label>
        <input type="number" name="Aptitude_Test_Score" required>

        <label>Soft Skills Rating</label>
        <input type="number" step="0.1" name="Soft_Skills_Rating"
required>
```

```
        <label>Certifications</label>
        <input type="number" name="Certifications" required>

        <label>Backlogs</label>
        <input type="number" name="Backlogs" required>

        <button type="submit">Predict Placement</button>


text }}
        </div>
    {% endif %}

</div>

</body>
</html>
```

App.py

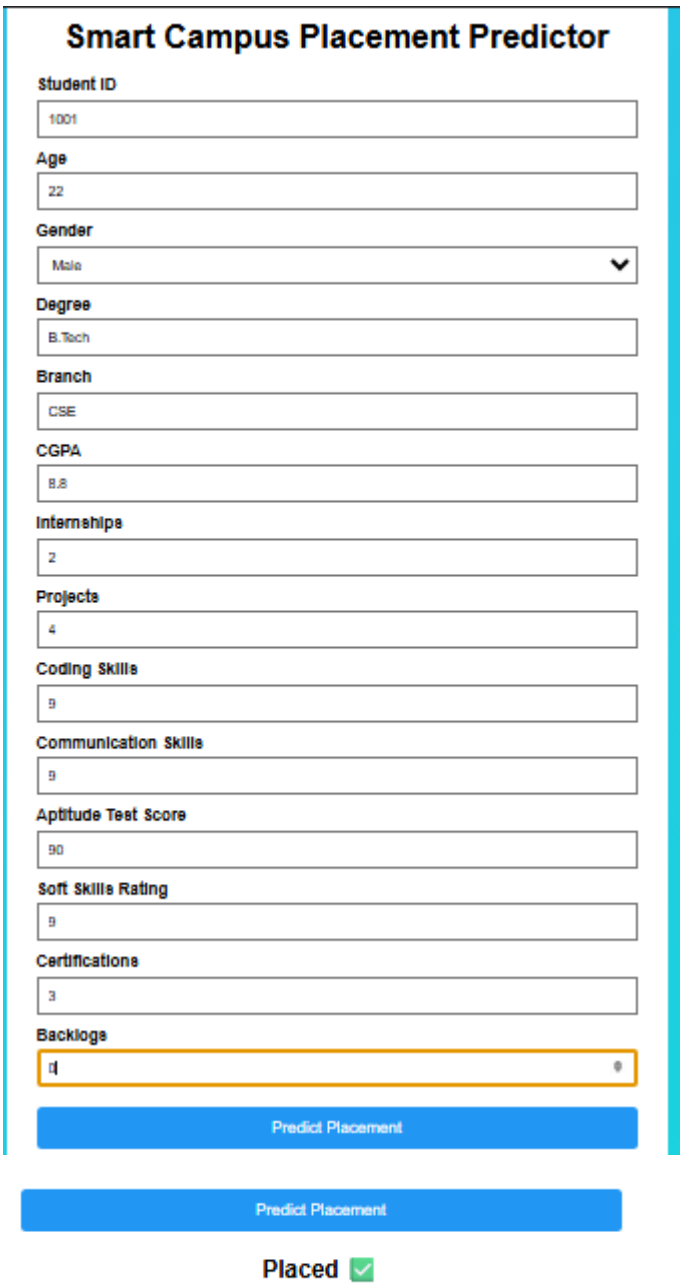
```
from flask import Flask, render_template, request
import pandas as pd
import pickle

app = Flask(__name__)

# Load model, scaler and encoders
model = pickle.load(open("placement_model.pkl", "rb"))
scaler = pickle.load(open("scaler.pkl", "rb"))
encoders = pickle.load(open("encoders.pkl", "rb"))


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():

    # Get values from form
    student_id = float(request.form["Student_ID"])
    age = float(request.form["Age"])
```

```
    gender = encoders["Gender"].transform(
        [request.form["Gender"]]
    )[0]

    degree = encoders["Degree"].transform(
        [request.form["Degree"]]
    )[0]

    branch = encoders["Branch"].transform(
        [request.form["Branch"]]
    )[0]

    cgpa = float(request.form["CGPA"])
    internships = float(request.form["Internships"])
    projects = float(request.form["Projects"])
    coding = float(request.form["Coding_Skills"])
    communication = float(request.form["Communication_Skills"])
    aptitude = float(request.form["Aptitude_Test_Score"])
    soft_skills = float(request.form["Soft_Skills_Rating"])
    certifications = float(request.form["Certifications"])
    backlogs = float(request.form["Backlogs"])

    # Create input data in exact dataset order
    data = [[
        student_id,
        age,
        gender,
        degree,
        branch,
        cgpa,
        internships,
        projects,
        coding,
        communication,
        aptitude,
        soft_skills,
        certifications,
        backlogs
    ]]

    # Convert to DataFrame
    data = pd.DataFrame(data, columns=[
```

```
        "Student_ID",
        "Age",
        "Gender",
        "Degree",
        "Branch",
        "CGPA",
        "Internships",
        "Projects",
        "Coding_Skills",
        "Communication_Skills",
        "Aptitude_Test_Score",
        "Soft_Skills_Rating",
        "Certifications",
        "Backlogs"
    ])

    # Scale input
    data = scaler.transform(data)

    # Prediction
    prediction = model.predict(data)[0]

    if prediction == 1:
        result = "Placed ✅"
    else:
        result = "Not Placed ❌"

    return render_template(
        "index.html",
        prediction_text=result
    )


if __name__ == "__main__":
    app.run(debug=True)
```

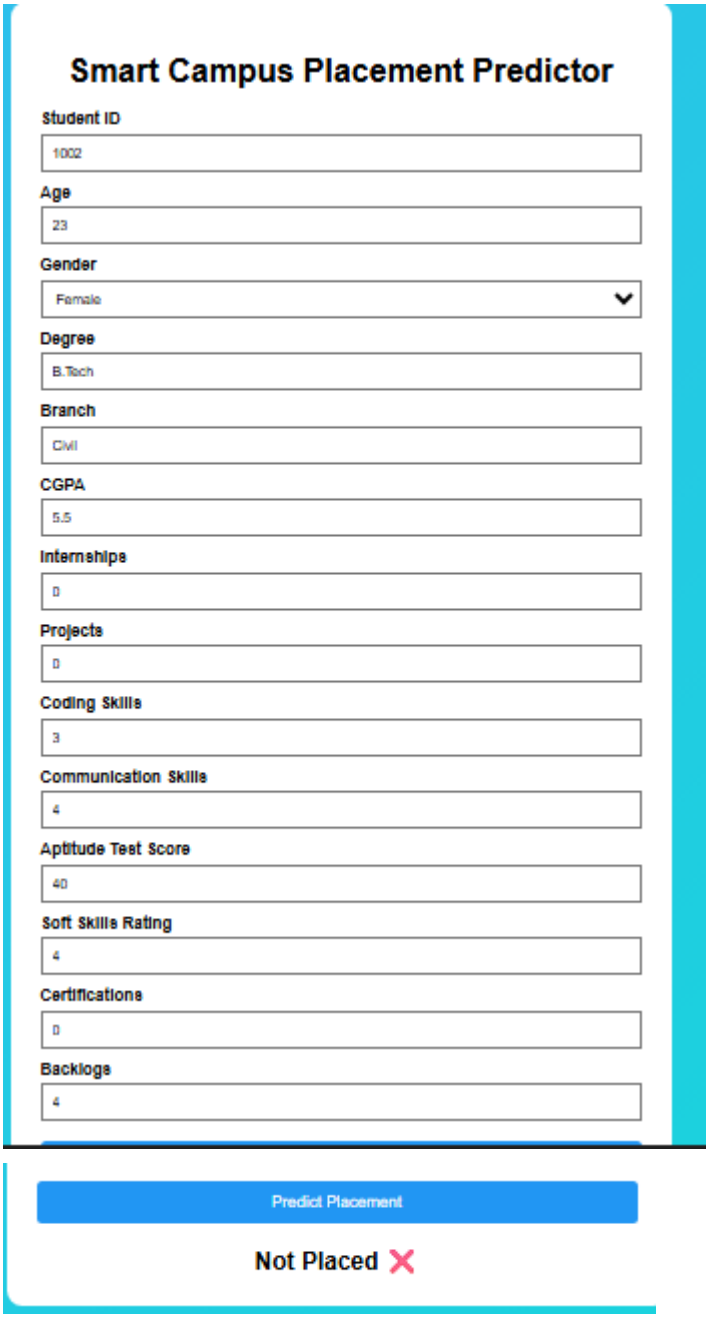
http://127.0.0.1:5000

Output 1 :-



Placed ✅

Output 2 ➖


Smart Campus Placement Predictor
Student ID
1002
Age
23
Gender
Female
Degree
B.Tech
Branch
Civil
CGPA
5.5
Internships
0
Projects
0
Coding Skills
3
Communication Skills
4
Aptitude Test Score
40
Soft Skills Rating
4
Certifications
0
Backlogs
4
Predict Placement
Not Placed